Verification Schemes: KZG Polynomial Commitments & Benabbas–Gennaro–Vahlis Delegation

Angel Coronel

Modern Cryptography – CUNY

May 19, 2025

Problem: How can a weak client efficiently verify work done by an untrusted server?

Motivation for schemes

- Protocols must let an untrusted server prove “I computed this correctly” faster than re-execution
- Tech does heavy analytics to the cloud but demand correctness
- Petabyte archives and blockchains need to keep costs down with sub-linear proof sizes
- Privacy laws and audits require verifiable integrity without revealing the data itslef
- Constant-size proofs are key for verifiers with tight latency and memory budgets

KZG Polynomial Commitments

1. Setup: Publish $(g, g^{\alpha}, \ldots, g^{\alpha^t})$.
2. Commit: $C = g^{\varphi(\alpha)}$ (constant size)
3. Open at z:
   - Server sends $y = \varphi(z)$
   - Witness $\pi = g^{(\varphi(X)-y)/(X-z)}\big|_{X=\alpha}$
4. Verify: Check $e(\pi, g^{\alpha-z}) = e(C, g)/e(g, g)^y$
   - The server really evaluated the committed polynomial at point z and the answer y is correct
5. Batch: One witness can open many points

Verifiable Delegation

- Pre-processing (once): Choose PRF key K and scalar r then mask coefficients $\tilde{A}(X) = \sum a_i F_K(i) X^i$ and upload to server
- Online Query at z
  - Server: Computes $y = \tilde{A}(z)$ and witness $\pi = g^{(\tilde{A}(\alpha)-y)/(\alpha-z)}$
  - Client:
    - Evaluates closed-form $R(z) = \sum F_K(i) z^i$ quickly
    - verifies $e(\pi, g^{\alpha-z}) = e(g^y, g) \cdot e\left(\prod g^{F_K(i)a_i}, g\right)^{r^{-1}}$
- Verification is private because of no trusted setup

Schemes Comparisons

	KZG Polynomial Commitments	**Verifiable Delegation**
Main Concept	Commit $C = g^{\varphi(\alpha)}$ - witness authenticates evaluations	Mask coefficients with algebraic PRF - leverage same pairing identity
Proof size	1 group element	1 group element
Verifier work	1–2 pairings ($\mathcal{O}(1)$)	1 pairing and PRF eval ($\mathcal{O}(1)$ or $\log n$)
Trusted setup	Yes - structured reference string SRS	No - standard parameters
Public / private	Publicly verifiable	Privately verifiable (needs key K)

Cryptographic Assumptions

- Bilinear groups G, G_T of prime order p with pairing $e : G \times G \to G_T$

Abbrev.	Informal Challenge	Used in
DL	Recover a from (g, g^a)	Baseline hardness for both
DDH	Distinguish (g^x, g^y, g^{xy}) from (g^x, g^y, g^z)	VD (log-time - base PRF)
t-SDH	Given $(g, g^\alpha, \ldots, g^{\alpha^t})$, output $(c, g^{1/(\alpha+c)})$	KZG binding
d-SDDH	Decisional variant of SDH	VD constant-time
Subgroup Membership	Distinguish a random element of G from one of $G_2 \subset G$	VD extension

KZG employs stronger t-SDH for constant proofs and public checks
VD lets the client trade assumption strength for speed

Security Guarantees

	KZG Polynomial Commitments	VD
Soundness / Binding	t-SDH ensures unique openings - forging witness breaks assumption	DDH / d-SDDH - forged (y, π) breaks PRF hardness
Hiding / Privacy	Commitment hides φ (DL) - reveals only y	PRF mask
Verifier type	Public	Private (key K)
Adaptive queries	Secure if SRS secret α remains hidden	Fresh blinding resists selective-failure attacks
Key-leak impact	Leaking α breaks all proofs	Exposure limited to that verifier's key

Efficiency

Metric	KZG Polynomial Commitments	VD
Trusted setup	SRS powers of α	None
Commitment / Storage	1 group element	Server stores $O(n)$ coefficients
Proof size per eval	1 group element	1 group element
Prover work	$O(t)$ (degree of polynomial)	$O(n)$ preprocessing - $O(1)$ per query
Verifier work	1–2 pairings ($O(1)$)	1 pairing and PRF eval ($O(1)$ or $\log n$)
Assumptions	t-SDH (and BSDH)	DDH - d-SDDH

Applications: KZG Polynomial Commitments

Domain	Description
Verifiable Secret Sharing	Constant-size commitments - one pairing witness per share
Zero-Knowledge Sets	Commit to vanishing polynomial - prove (non-)membership
Blockchain Roll-ups	Succinct data availability proofs
Selective-Disclosure Credentials	Compact attribute proofs for selective field release

Applications: Verifiable Delegation

Domain	Description
Proofs of Retrievability	Treat file as coefficients - one pairing + PRF per audit
Verifiable Keyword Search	Roots encode keywords - server proves query privately using PRF-masked values
Verifiable Databases	Composite-order variant gives $O(1)$ updates without extra machinery

When to Use Which

KZG Polynomial Commitments

- Public verification - good for open audit trails
- Constant-size proofs & verifier time
- Requires trusted setup - stronger t-SDH assumption
- Leak of α is bad

VD

- No ceremony - pairing parameters
- Verifier can tune assumption based on CPU cost
- Proofs privately verifiable (needs key K)
- Adaptive security - leakage local to client only

Recommend to use *KZG* for public, transparent audits - use *VD* for massive private datasets that dislike ceremonies

Conclusion

- Both schemes have same algebraic foundation but different philosophies
  - KZG is public & ceremony-bound —— VD is private & setup-free
- Main trade off is trust vs transparency
- Concise proof research is moving towards constant-size, transparent, publicly verifiable commitments
- Hybrids can be made to get best of both worlds blending setup-free scalability and have public accountability

Questions?